



汇编程序设计课程实验报告

题目 子程序设计 1

姓 名：王子琪

(学 号)：22920182204309

院 系：信息学院

专 业：人工智能

年 级：2018 级

2020 年 5 月 5 日

一、实验目的

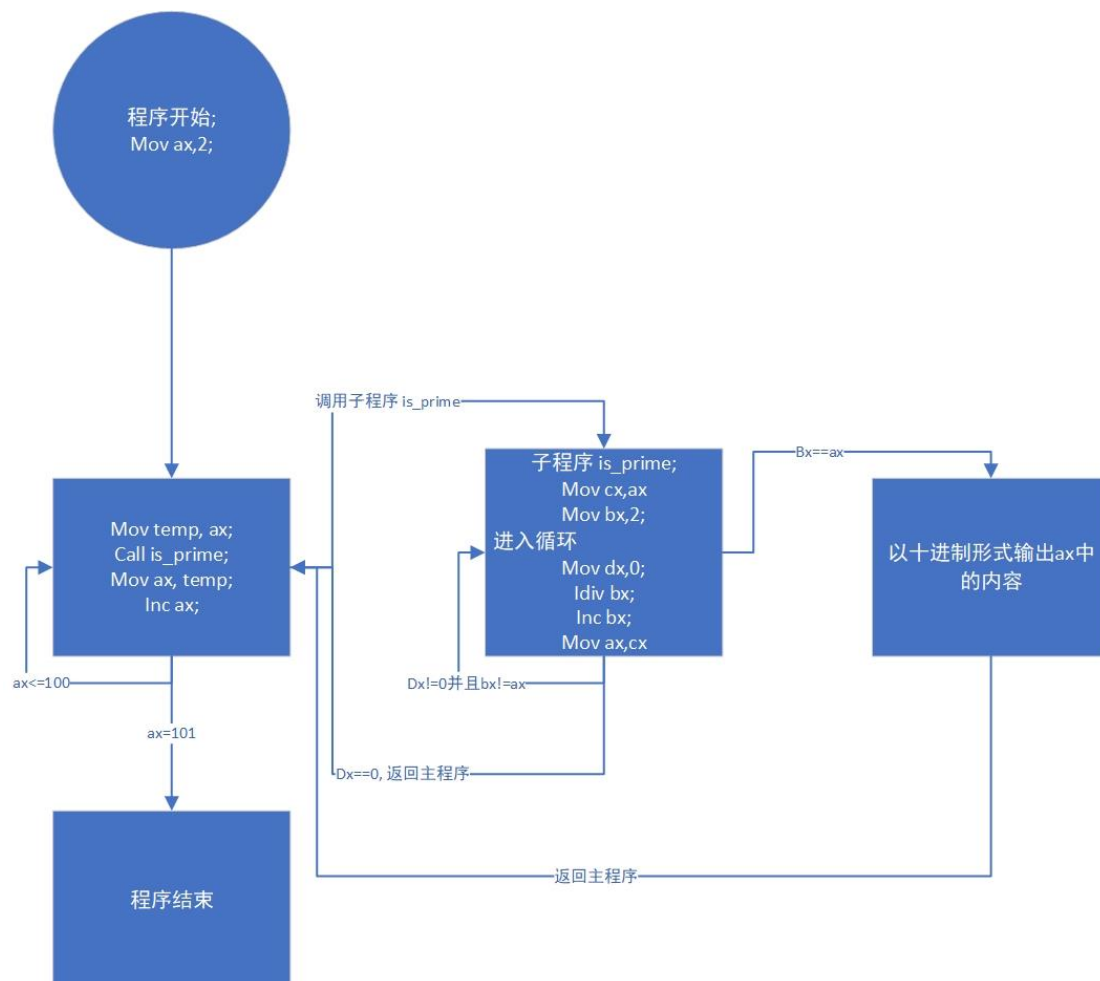
1. 熟悉汇编语言程序设计结构
2. 熟悉汇编语言程序子程序设计方法
3. 了解汇编语言程序设计参数传递方法

二、实验思路

1 编写一个判断是否为素数的子程序，在主程序中调用该子程序，并在屏幕中显示 1~100 的所有素数

思路，从 2-100 逐个判断，分别再判断是否为素数，和输出的方面，调用子程序，其中输出部分可以参考实验五。

流程图：



代码:

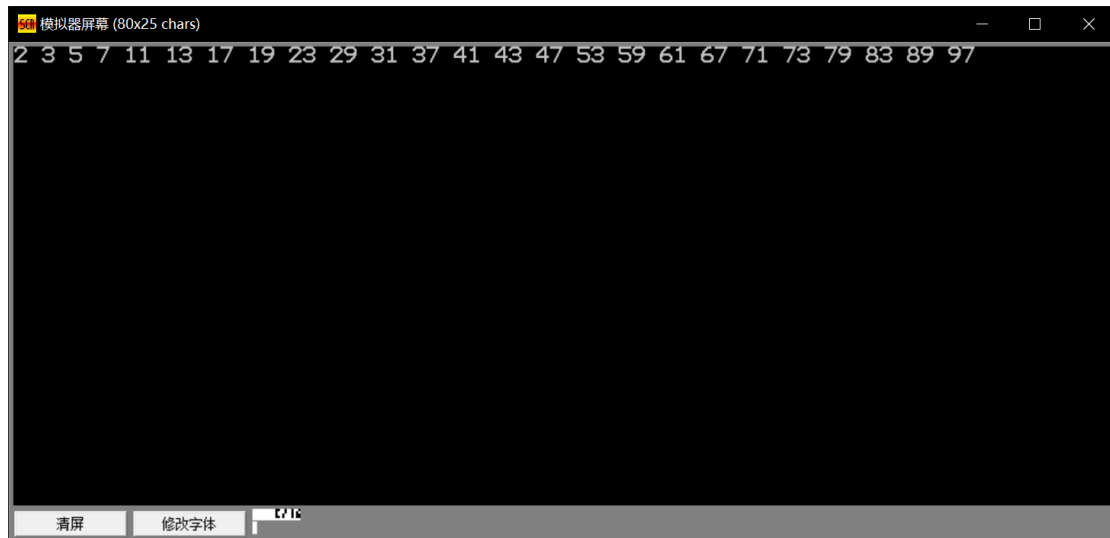
```
001 ; multi-segment executable file template.
002
003 data segment
004     ; add your data here!
005     k dw 10
006     mess1 db 2 dup(0)
007     temp dw 0
008 ends
009
010 stack segment
011     dw 128 dup(0)
012 ends
013
014 code segment
015 start:
016 ; set segment registers:
017     mov ax, data
018     mov ds, ax
019     mov es, ax
020
021 main proc near
022     mov ax, 2
023 loop1:
024     mov temp, ax
025     cmp ax, 101
026     jz exit
027     call is_prime
028     mov ax, temp
029     inc ax
030     jmp loop1
031 exit:
032     ; wait for any key...
033     mov ah, 1
034     int 21h
035
```

```

036     mov ax, 4c00h ; exit to operating system.
037     int 21h
038 main endp
039
040
041 is_prime proc near
042     mov bx, 2;
043     mov dx, 0
044     mov cx, ax
045 loop2:
046     mov dx, 0
047     cmp bx, ax
048     jz prin
049     idiv bx
050     cmp dx, 0
051     jz exit1
052     mov ax, cx
053     inc bx
054     jmp loop2
055 prin:
056     call print
057 exit1:
058     ret
059 is_prime endp
060
061
062 print proc near
063 ;转换十六进制为十进制，并以字符形式存储在mess1中
064     mov cx, 0
065     lea di, mess1
066     mov dx, 0
067 loop3:
068     cmp ax, 0
069     jz cout ;脱离循环的为一条条件
070     div k ;
071     add dl, 48 ;dl中存储的是余数，将数字形式转换为字符形式存储到首地址为mess1的字符串中
072     mov es:[di], dl
073     mov dl, 0
074     inc di ;字符串指针移向下一个空位
075     inc cx ;代表十进制位数多1
076     jmp loop3
077 ;倒序逐个输出mess1中的内容
078 cout:
079     dec di ;此时di实际指向在有意义的字符的下一位，所以要减一
080 loop4:
081     cmp cx, 0 ;cx是之前创建的计数器
082     jz exit2
083     mov dl, es:[di] ;逐位输出
084     mov ah, 02h
085     int 21h
086     dec cx ;
087     dec di
088     jmp loop4
089 exit2:
090     mov dl, ' '
091     mov ah, 02h
092     int 21h
093
094     ret
095 print endp
096
097 ends
098
099 end start ; set entry point and stop the assembler.

```

结果;



2. 英文人名排序。从终端输入 10 个人名，当所有人名都键入后，按字典序将人名排序，并在屏幕上显示已经排好序的人名。

要求：按功能分模块编写子程序实现

名字案例：

Tom

Tomson

Jerry

Jan

Wangziqui

Alliy

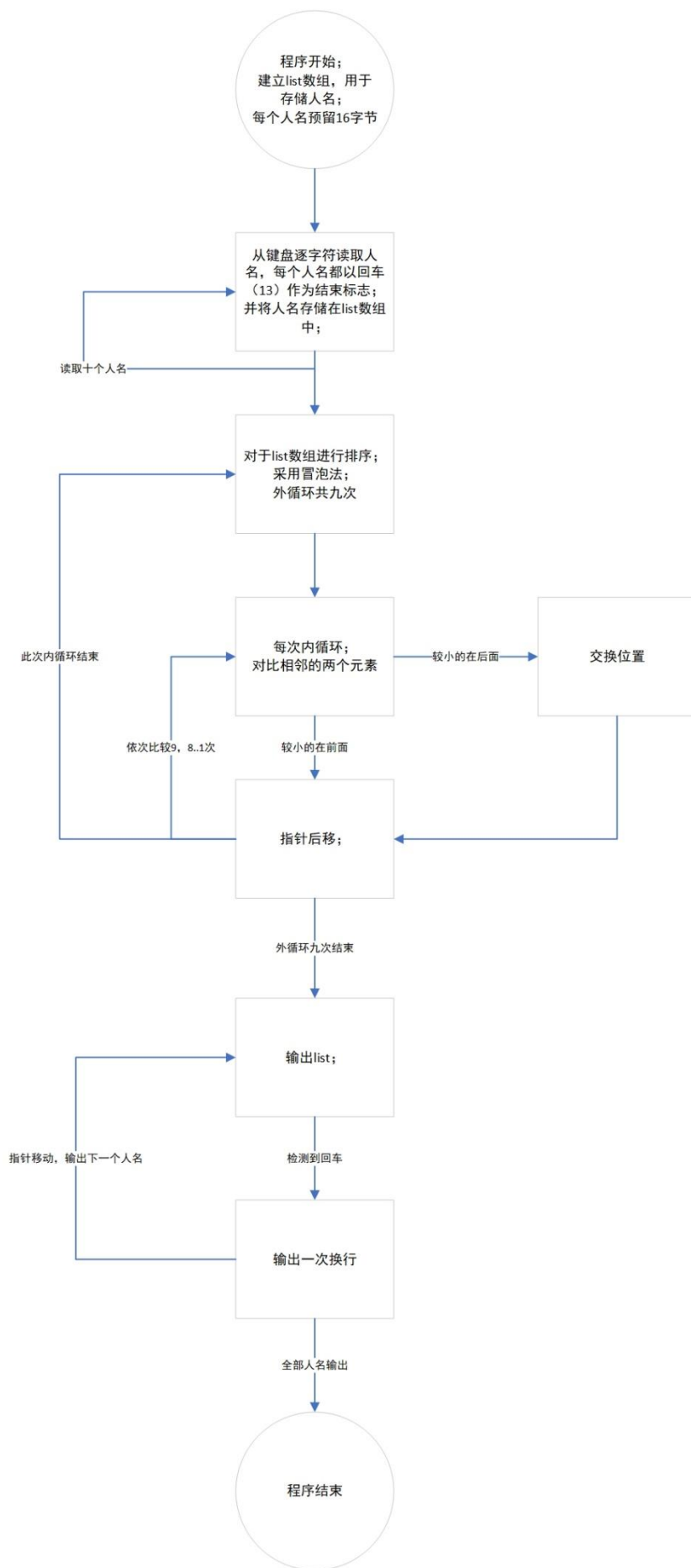
Marry

Mark

Havid

Harson

流程图



代码:

```
001 ; multi-segment executable file template.
002
003 data segment
004     ; add your data here!
005     list db 160 dup(0) ;负责存出人名, 每个人名预留15字节, 人名以回车(13)结束
006     temp db 16 dup(0) ;临时缓冲区, 在人名排序时, 作为一个中间量, 帮助传递
007     temp1 dw 0 ;临时存储区, 用于存储地址, 在调用子程序前, 存入, 子程序结束读出, 不用堆栈是因为, 子程序需要用到地址数据
008 ends
009
010 stack segment
011     dw 128 dup(0)
012 ends
013
014 code segment
015 start:
016 ; set segment registers:
017     mov ax, data
018     mov ds, ax
019     mov es, ax
020
021 main proc near ;主程序
022     call cin ;负责从键盘读取人名
023     call order ;负责排序
024     call prin ;负责输出
025     ; wait for any key....
026     mov ah, 1
027     int 21h
028
029     mov ax, 4c00h ; exit to operating system.
030     int 21h
031 main endp
032
033 cin proc near ;读取采用01号中断和循环叠加的情况(认为空格也作为人名一部分参与排序)
034     mov cx, 0
035
036     mov di, 0
037 loopcin1:
038     add cx, 16 ; cx中存储的是, 下一次人名的起始地址
039 loopcin2:
040     mov ah, 01
041     int 21h
042     mov list[di], al
043     cmp al, 13 ;每当输入了一个回车, 就认为这个人名已经输完
044     jz exitcin
045     inc di
046     jmp loopcin2
047 exitcin:
048     ; cx中存储的是, 下一次
049     cmp cx, 160 ;此时意味已经读完了十个人名
050     jz finsh
051     mov di, cx ;没有读完的话, di指针指向下一个人名开始的相对地址
052     jmp loopcin1
053 finsh:
054     ret
055 cin endp
056
057 order proc near
058     mov cx, 10
059     mov di, cx ;使用di来临时存储cx的下一轮初始值+1, 同时控制外循环是否进行
060 orloop1:
061     mov cx, di
062     dec cx
063     cmp cx, 0
064     jz orderexit ;倘若此时cx的值为零, 也就是外循环次数di的值为1, 则外循环已经结束了9次循环
065     mov di, cx ;使用di来临时存储cx的下一轮初始值+1, 同时控制外循环是否进行
066     mov bx, 0 ;bx存储的是相对于首地址的位移量, 每一次外循环都是从数组头开始
067 orloop3:
068     cmp cx, 0 ;内循环的作用是, 将最大的数压到最底下,
069     jz ornext ;倘若cx为零表示此次内循环结束
```

```

068     mov temp1,bx      ; 临时存储, 避免因子程序的改动, 影响这里的循环
069     call compare      ; 调用子程序, 负责比较这个时候, bx的指向和bx+16指向的人名的大小关系, 并用ax返回比较结果
070     mov bx,temp1
071     cmp ax,1
072     jz orcontinue    ; 说明较小的数字在较前的位置, 所以无须交换
073     mov temp1,bx
074     call exchange    ; 说明较大的数字在较前的位置, 前后交换, list[bx]和list[bx+16]交换位置
075     mov bx,temp1
076 orcontinue:
077     add bx,16
078     dec cx
079     jmp orloop3      ; 继续内循环
080 ornext:
081     jmp orloop1      ; 继续外循环
082 orderexit:
083     ret
084 order endp
085
086
087 compare proc near
088     push cx
089     mov ax,0
090     mov cx,16
091 comloop:
092     cmp cx,0          ; 比较这两个人名
093     jz comno:         ; 倘若cx为零, 说明两个人名完全相同
094     mov al,list[bx]   ; 逐个比较人名中的各个字符
095     cmp al,list[bx+16]
096     jz comcontin     ; 如果相等, 则继续比较
097     js comno         ; 如果地址靠前的数字较小, 则ax标记1
098     jmp comyes       ; 如果地址靠后的数字较小, 则ax标记0
099 comcontin:
100     inc bx
101
102     dec cx
103     jmp comloop
104 comno:
105     mov ax,1
106     jmp comexit
107 comyes:
108     mov ax,0
109     jmp comexit
110 comexit:
111     pop cx
112     ret
113 compare endp
114
115 exchange proc near
116     push cx
117     push si
118     push dx
119     push ax
120
121     mov cx,16
122     mov ax,bx          ; ax负责临时存储bx的地址, 总共有三次交换, 而每次交换前, 回复bx的初始值
123     mov si,0           ; si是temp的指针
124     mov dx,list[bx]    ; list[bx]移入temp
125     cmp cx,0
126     jz excont1
127     mov dl,list[bx]
128     mov temp[si],dl
129     inc bx
130     inc si
131     dec cx
132     jmp exloop1
133 excont1:
134     mov cx,16

```

```

134     mov bx, ax
135 exloop2:
136     cmp cx, 0           ; list[bx+16]移入list[bx]
137     jz excont2
138     mov dl, list[bx+16]
139     mov list[bx], dl
140     inc bx
141     dec cx
142     jmp exloop2
143 excont2:
144     mov cx, 16
145     mov bx, ax
146     mov si, 0
147 exloop3:                ; temp移入list[bx+16]
148     cmp cx, 0
149     jz exexit
150     mov dl, temp[si]
151     mov list[bx+16], dl
152     inc bx
153     inc si
154     dec cx
155     jmp exloop3
156 exexit:
157     mov bx, ax
158
159     pop ax
160     pop si
161     pop dx
162     pop cx
163     ret
164 exchange endp
165
166

```

```

167 prin proc near          ;输出list中的内容，采用02号中断和循环配合的方法
168     push cx
169     push di
170     push bx
171     push dx
172     mov bx, 0
173     mov cx, 10           ;总计输出10个人名
174     mov di, 0
175     add bx, 16           ;bx中负责存储，下一个人名的相对地址
176
177     mov dl, 10           ;先回车换行一下，和输入区分开来
178     mov ah, 02
179     int 21h
180     mov dl, 13
181     mov ah, 02
182     int 21h
183 prinloop:               ;
184     cmp cx, 0
185     jz prinexit
186     mov dl, list[di]     ;调用02号中断输出
187     cmp dl, 13
188     jz prinnext          ;每个人名都是用回车（13）作为结尾
189     mov ah, 02
190     int 21h
191     inc di
192     jmp prinloop
193 prinnext:               ;一个人名输出结束
194     mov dl, 10           ;回车换行一下
195     mov ah, 02
196     int 21h
197     mov dl, 13
198     mov ah, 02
199     int 21h

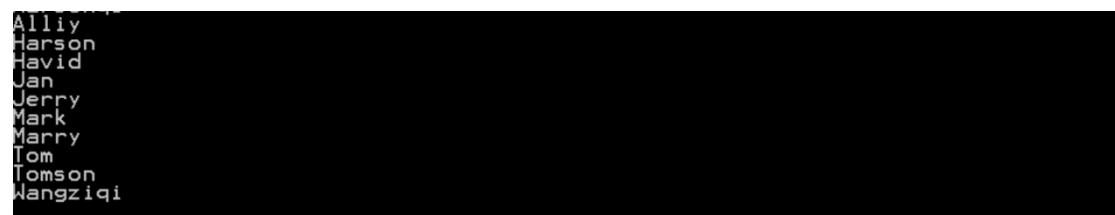
```

```

200     dec cx
201     mov di, bx           ;di指针指向下一个人名
202     add bx, 16
203     jmp prinloop
204 prinexit:
205     pop si
206     pop dx
207     pop bx
208     pop dx
209     pop cx
210     ret
211 prin endp
212
213 ends
214
215 end start ; set entry point and stop the assembler.
216

```

结果：



```
Alliy
Harson
Havid
Jan
Jerry
Mark
Marry
Tom
Tomson
Wangziqu
```

四、实验心得

收获：

对于子程序有了更深的理解。

对于循环，和分支有了更深的理解

对于中断输入输出有了更深的把握

对于大型程序的搭建有了初步的尝试

对于汇编的理解更加深入。

不足：

对于具体指令的记忆

大规模程序容易考虑不周，遗漏细节